

EDUGRAPH AI

School Box 40-Day Execution Guide

8-Week Pilot Implementation Plan

A day-by-day delivery roadmap for curriculum ingestion, exam-CLO matching, calibrated gap analysis, and offline edge deployment in Ethiopian pilot schools.

01

Curriculum Ingestion

Graph decomposition and curriculum mirroring

02

Exam-CLO Matching

Quantized embeddings and teacher verification

03

Gap Analysis

Graph traversal, local LLM, and study plans

PILOT SCOPE

2-3 schools | 8 weeks | 40 working days | Offline-first School Box architecture

Executive Overview

Here is the comprehensive, 40-day (Day-by-Day) execution guide for the 8-week pilot launch. Every single day specifies exact backend tasks, AI/ML workflows, frontend updates, and deliverable checkpoints across your three core pillars (Curriculum Ingestion, Exam-CLO Matching, Gap Analysis) and the School Box edge server.

Program Roadmap

Month	Weeks	Primary Focus	Outcome
1	1-4	Core engine calibration and edge capabilities	Three hardened pillars, tenant isolation, and sync foundation
2	5-8	Packaging, simulation, installation, and rollout	Deployable School Boxes, trained staff, and live pilot operations

Month 1: Core Engine Calibration & School Box Edge Capabilities

Core technical work and operational checkpoints for this phase.

Week 1: Pillar 1 (Curriculum Ingestion) & Local Storage Setup

Day 1 (Mon): Development Environment & Magic-Byte File Validation Backend (Go): Audit backend/cmd/api/main.go and router.go. Implement magic-byte inspection (sniffCurriculumMime) in backend/internal/curriculum/handler/handler.go to inspect the first 512 bytes of uploaded documents. Block MIME spoofing for .pdf (%PDF-) and .docx (PK\x03\x04) regardless of Content-Type headers. AI Service (Python): Verify local Docker Compose container wiring for api, ai-service, postgres, neo4j, redis, and ollama. Frontend: Add MIME validation error toasts in UploadPage.tsx. Deliverable Checkpoint: Invalid file types are rejected with a 400 Bad Request prior to queue insertion.

Day 2 (Tue): Local Storage Adapter & Multi-Column Parser Hardening Backend (Go): Enforce PostgresStorage (storage.local_files BYTEA) as the primary storage provider in backend/pkg/storage/postgres.go for local School Box deployment. AI Service (Python): Update app/services/curriculum_parser/extractor.py and docx_extractor.py. Implement bounding-box x-coordinate clustering in PyMuPDF to process multi-column syllabus PDFs without interleaving text. Frontend: Connect the file upload dropzone to POST /api/v1/curriculum/upload. Deliverable Checkpoint: Multi-column PDF curriculum documents parse into clean, sequential text blocks.

Day 3 (Wed): Queue Wiring & Hierarchical Structure Extraction Backend (Go): Connect curriculum upload handlers to push job UUID strings to queue:curriculum:parse via Redis LPUSH. AI Service (Python): Refine curriculum_worker.py to process background jobs via BRPOP. Parse text headers using regex rules into a structured JSON hierarchy (Subject -> Unit -> Topic -> CLO) and save to curriculum.upload_jobs.parsed_structure. Frontend: Build polling/websocket status indicator on JobReviewPage.tsx tracking pending -> parsing -> parsed states. Deliverable Checkpoint: Uploaded curriculum file generates a valid parsed_structure JSONB tree in Postgres.

Day 4 (Thu): Curation UI & Authenticated Document Previews
Backend (Go): Expose GET `/api/v1/storage/files/{jobId}` proxy endpoint to return document raw bytes.
AI Service (Python): Add robust JSON schema validation on parsed trees to prevent missing unit/topic titles.
Frontend: Enhance `JobReviewPage.tsx` in `frontend/src/features/curriculum/`. Add an interactive tree view allowing officers to edit titles, re-order topics, and delete misparsed CLOs. Implement blob-fetching with JWT Bearer headers for the embedded document viewer.
Deliverable Checkpoint: Officers can view the source file and edit the extracted tree hierarchy in the UI.

Day 5 (Fri): Relational Promotion & Neo4j DAG Mirroring
Backend (Go): Implement `ApproveAndPromote` in `backend/internal/curriculum/repository/repository.go`. Run a single SQL transaction inserting `curriculum.subjects`, `units`, `topics`, and `clos`. Execute post-commit Neo4j Cypher mutations to create `(:Subject)-[:HAS_UNIT]->(:Unit)-[:HAS_TOPIC]->(:Topic)`.
AI Service (Python): Verify Neo4j connection pool resilience during batch node creations.
Frontend: Add approval confirmation modal and redirect to curriculum tree dashboard.
Deliverable Checkpoint: Approved curriculum appears in both Postgres relational tables and Neo4j graph nodes.

Week 2: Pillar 2 (Exam-CLO Matching) & Quantized CPU Embeddings

Day 6 (Mon): Exam Parsing & Question Block Extraction
Backend (Go): Build POST `/api/v1/exams/upload` endpoint in `internal/assessment/`. Push job payload to Redis queue `queue:exam:parse`.
AI Service (Python): Implement `exam_worker.py` to parse exam PDFs and DOCX files. Extract individual question stems, option sets (A, B, C, D), allocated marks, and estimated answer times into `assessment.questions`.
Frontend: Build `ExamUploadPage.tsx` with document drag-and-drop.
Deliverable Checkpoint: Exam document is parsed into discrete question records in Postgres.

Day 7 (Tue): Answer Key Alignment & Metadata Schema Wiring
Backend (Go): Expose endpoint POST `/api/v1/exams/{id}/answer-key`.
AI Service (Python): Build `answer_key_worker.py` to parse uploaded answer keys (`queue:exam:answerkey`). Align question numbers, correct options, and detailed solutions to corresponding question IDs.
Frontend: Add answer-key mapping UI to review parsed answer choices against extracted questions.
Deliverable Checkpoint: Parsed questions have mapped correct choices and point allocations.

Day 8 (Wed): BGE-M3 ONNX Quantization for School Box CPU AI Service (Python): Convert BAAI/bge-m3 embedding model to ONNX INT8 format (bge-m3-int8.onnx). Create embedding wrapper using onnxruntime to run dense vector generation on low-power CPUs without GPU dependencies. **Backend (Go):** Ensure vector schemas exist (vector(1024) in curriculum.clos and assessment.questions). **Deliverable Checkpoint:** ONNX model generates 1024-dim embeddings locally on CPU in under 15ms per text string.

Day 9 (Thu): Semantic Vector Matching Engine Construction AI Service (Python): Build cosine similarity matching logic between question embeddings and CLO embeddings. Implement threshold scoring: $\text{Score} \geq 0.85 \implies \text{Auto-Accept}$, $0.65 \leq \text{Score} < 0.85 \implies \text{Review Flag}$, $\text{Score} < 0.65 \implies \text{Reject}$. **Backend (Go):** Expose GET /api/v1/exams/{id}/clo-matches endpoint to retrieve matched pairs. **Frontend:** Build match confidence badge indicators (High, Needs Review, Low) on ExamReviewPage.tsx. **Deliverable Checkpoint:** System auto-aligns exam questions to curriculum CLOs with similarity scores.

Day 10 (Fri): Teacher Verification UI & Exam Publication Backend (Go): Build POST /api/v1/exams/{id}/verify endpoint to accept modified CLO mappings and publish the exam. **Frontend:** Finalize ExamReviewPage.tsx allowing teachers to manually select CLOs from a dropdown for flagged questions. **Deliverable Checkpoint:** Teacher confirms/edits mappings, flipping exam status to published for student access.

Week 3: Pillar 3 (Calibrated Gap Analysis Engine) & Local LLM

Day 11 (Mon): Exam Submissions & Exposure Tracking Backend (Go): Update exam_submit.go (POST /api/v1/exams/{id}/submit) to store per-question student answers and timeSpentSecs. Increment exposure_count on student-topic relations in students.gap_records. Push job to queue:gap:analyze. **Frontend:** Update TakeExamPage.tsx to record per-item answer durations. **Deliverable Checkpoint:** Student submission triggers an async gap analysis job while updating topic exposure metrics.

Day 12 (Tue): Cypher Root-Cause Traversal Engine AI Service (Python): Implement gap_worker.py in app/services/gap_analysis/. Execute 3-pass analysis: **Triage:** Find wrong answers and unmastered CLOs. **Graph Walk:** Traverse Neo4j (:Topic)-[:HAS_PREREQUISITE*1..3]->(:Topic). **Filtering:** Require exposure_count >= 1 on root nodes to classify as confirmed root causes. **Deliverable Checkpoint:** Engine isolates root-cause topics across grade levels using graph traversal.

Day 13 (Wed): Guess Discounting & Untested Categorization
AI Service (Python): Add MCQ 25% base guess discounting logic to single-attempt submissions. Flag upstream topics with `exposure_count == 0` as "Unverified Prerequisite Gaps" in output records. **Backend (Go):** Save structured outputs to `students.gap_records` and `students.exam_insights`. **Frontend:** Display "Unverified" badges vs "Confirmed Gap" warnings in student insight cards. **Deliverable Checkpoint:** Guessing is discounted and untested prerequisites are visually separated from true gaps.

Day 14 (Thu): Local Ollama Integration for Offline Insights
AI Service (Python): Configure local Ollama integration in `app/services/gap_analysis/llm.py` running `qwen2.5:3b-instruct` or `llama3.2:3b`. Synthesize Cypher root-cause chains into bilingual diagnostic text (English and Amharic). **Frontend:** Render diagnostic summaries in student results view (`/exams/{id}/results/page.tsx`). **Deliverable Checkpoint:** Student receives encouraging, bilingual diagnostic feedback without cloud connectivity.

Day 15 (Fri): Topological Study Plan Generation
AI Service (Python): Build `study_plan_worker.py` listening on `queue:studyplan:generate`. Run Kahn's topological sort over prerequisite DAGs to order daily study blocks, placing root causes before downstream symptoms. **Backend (Go):** Expose `GET /api/v1/students/me/study-plans`. **Frontend:** Render calendar roadmap view in `frontend/src/app/(student)/study-plan/page.tsx`. **Deliverable Checkpoint:** Automated study plan generates daily learning targets structured by topological dependencies.

Week 4: Multi-Tenant Security, RBAC & Outbox Sync Pattern

Day 16 (Mon): Multi-School Tenant Boundaries & Scoping
Backend (Go): Enforce `school_id` filtering across SQL repositories (`assessment`, `student`, `curriculum`). Update Neo4j Cypher queries to enforce school scoping: `CypherMATCH (sch:School {id: $school_id})-[:ENROLLS]->(s:Student {id: $student_id})`

DELIVERABLE CHECKPOINT SQL and Cypher queries strictly prevent cross-school data visibility.

Day 17 (Tue): Offline JWT Leases & Asymmetric Signatures
Backend (Go): Configure RS256 JWT key management (`APP_ENV=production_edge`) in `backend/pkg/crypto/crypto.go`. Issue 14-day offline refresh token leases stored in local Postgres/Redis to allow off-grid logins. **Frontend:** Handle offline session persistence in `auth.store.ts`. **Deliverable Checkpoint:** Users can log in and refresh JWT tokens on a disconnected server for up to 14 days.

Day 18 (Wed): Outbox Store-and-Forward Pattern SetupBackend (Go): Create `public.sync_logs` and `public.outbox_events` tables. Update `exam_submit.go` to insert an outbox JSON payload in the same SQL transaction as the student submission. **Deliverable Checkpoint:** All local student activity generates structured outbox event logs for future sync.

Day 19 (Thu): Class-Wide Gap Heatmap AggregationBackend (Go): Implement `GET /api/v1/teachers/me/class-heatmap` using Cypher graph queries. Trigger cross-grade warnings when $>40\%$ of a class exhibits shared unmastered prerequisite gaps. **Frontend:** Render heatmap grid using `HeatmapGrid.tsx` in `TeacherDashboardPage.tsx`. **Deliverable Checkpoint:** Teacher dashboard highlights cohort-wide prerequisite bottlenecks.

Day 20 (Fri): School Quality Metrics & Redis CachingBackend (Go): Implement Redis caching (`school_quality:{schoolId}`, TTL 1h) for school quality scores. Verify background worker in `internal/assessment/qualityworker` runs without memory leaks. **Deliverable Checkpoint:** Composite school quality scores compute efficiently with 1-hour cache invalidation.

Month 2: School Box Packaging, Simulation & Field Rollout

Core technical work and operational checkpoints for this phase.

Week 5: School Box OS, Containerization & Hardware Staging

Day 21 (Mon): Production Docker Compose Configuration
 DevOps: Create docker-compose.schoolbox.yml with resource constraints:
 Postgres: 2 CPU cores, 4GB RAM
 Neo4j: 2 CPU cores, 4GB RAM
 Python AI Service: 2 CPU cores, 4GB RAM
 DevOps: Set explicit volume mounts for local data persistence and restart: always container policies.
 Deliverable Checkpoint: Docker stack spins up reliably under bounded memory and CPU limits.

Day 22 (Tue): Network Cable Unplug Isolation Test
 Testing: Physically disconnect WAN/4G cables from the test server.
 Verification: Test local user authentication, exam taking, gap analysis generation, topological study plan rendering, and Graph-RAG AI Tutor interactions using local Ollama. Deliverable Checkpoint: Entire application stack functions offline with zero external internet dependencies.

Day 23 (Wed): Automated USB Backup Cron Automation
 DevOps: Write shell backup script (schoolbox-backup.sh) executing pg_dump for Postgres and neo4j-admin database dump. Configure systemd cron task to run nightly at 01:00 AM, writing encrypted archives to an attached external USB drive. Deliverable Checkpoint: Automated local backups execute cleanly to external media.

Day 24 (Thu): Physical Server Hardware Flashing
 DevOps: Install Ubuntu Server 24.04 LTS on 3 physical mini-PCs ("School Boxes"). Configure static local IP addresses, systemd Docker auto-start, and local DNS mapping (http://schoolbox.local). Deliverable Checkpoint: 3 physical hardware nodes are imaged, networked, and ready for deployment.

Day 25 (Fri): Ethiopian Ministry Curriculum Pre-Seeding
 Database: Run seed_curriculum.py to pre-seed Grade 9-12 Ethiopian Ministry curricula (Math, Physics, Chemistry) into local Postgres and Neo4j database volumes on all 3 hardware units. Deliverable Checkpoint: School Boxes contain full K-12 national curriculum DAGs out-of-the-box.

Week 6: LAN Stress Testing, Sync Dry-Run & Polish

Day 26 (Mon): 150-Device LAN Stress Test Testing: Set up local Wi-Fi router testbed. Execute k6 load testing scripts simulating 150 concurrent student exam submissions over local Wi-Fi. Monitoring: Profile CPU load, RAM usage, and Redis queue processing rates (queue:gap:analyze). Deliverable Checkpoint: System handles peak local exam submission bursts without dropping connections or dropping jobs.

Day 27 (Tue): Store-and-Forward Sync Dry-Run Backend (Go): Build sync worker (internal/sync) that reads un-synced events from public.outbox_events. Testing: Simulate 3 days of offline activity. Connect a 4G dongle and trigger sync worker to push batched events to the Central Cloud. Deliverable Checkpoint: Local outbox records sync successfully to Central Cloud upon network reconnection.

Day 28 (Wed): Idempotency & UUID Collision Testing Backend (Go): Verify database writes use UUIDv4 primary keys across all entities. Test duplicate sync payload deliveries to ensure handlers process events idempotently without throwing primary key conflicts. Deliverable Checkpoint: Re-sent sync payloads process cleanly without duplicating records.

Day 29 (Thu): Local Admin Health Portal Frontend: Build lightweight local health dashboard at /admin/school-box. Display disk utilization, local Redis queue status, CPU temperature, and outbox sync logs. Deliverable Checkpoint: Local school admins can visually check hardware health and sync status.

Day 30 (Fri): Responsive UI Polish & Amharic Translations Frontend: Polish mobile and tablet responsive layouts for lower-cost student devices. Verify Amharic translation strings across dashboards, error toasts, and AI diagnostic components. Deliverable Checkpoint: UI renders cleanly across tablets with accurate English and Amharic labels.

Week 7: On-Site Installation, School Setup & Staff Training

Day 31 (Mon): On-Site Installation (School 1 & School 2) Field Ops: Transport and mount physical School Box servers in computer labs at School 1 and School 2. Connect Ethernet cables to local lab switches and verify LAN routing ([http://schoolbox.local:8080])(http://schoolbox.local:8080)). Deliverable Checkpoint: School Boxes 1 and 2 are installed and reachable from student lab PCs/tablets.

Day 32 (Tue): On-Site Installation (School 3) & Network Sanity Checks
Field Ops: Install physical School Box server in School 3. Conduct network connectivity tests across all client devices in the lab.
Deliverable Checkpoint: All 3 pilot schools have active, reachable School Box hardware nodes.

Day 33 (Wed): Teacher Training - Exam & Matching Workflow
Training: Run interactive training for teachers: uploading paper exam documents, reviewing auto-matched CLOs, and publishing digital tests.
Deliverable Checkpoint: Teachers successfully create and publish test exams on the local server.

Day 34 (Thu): Teacher & Admin Training - Heatmaps & Remediation
Training: Train teachers on viewing Class-Wide Gap Heatmaps and assigning remedial study plans. Train school admins on monitoring local server health and managing user rosters.
Deliverable Checkpoint: Staff demonstrate proficiency in navigating diagnostic heatmaps and user administration.

Day 35 (Fri): Student CSV Roster Bulk Import
Data Ops: Execute bulk CSV user imports to create student accounts, linking them to their corresponding school, grade, and section records.
Deliverable Checkpoint: Student accounts are provisioned and ready for launch day logins.

Week 8: Field Go-Live & Operational Support

Day 36 (Mon): OFFICIAL PILOT GO-LIVE
Deployment: Launch EduGraph AI across all 2-3 pilot schools. Students log into local School Box web portals during scheduled lab sessions.
Field Ops: On-site engineers monitor initial user traffic and server loads.
Deliverable Checkpoint: Pilot is live with active student logins across all 3 schools.

Day 37 (Tue): Live Digital Exam Session Monitoring
Field Ops: Supervise first live digital exam administration. Monitor real-time background Redis queue processing (queue:gap:analyze) on local School Boxes.
Deliverable Checkpoint: Live student exam submissions process smoothly into gap records without delay.

Day 38 (Wed): Diagnostic Accuracy & Gap Output AuditQA: Review generated students.gap_records and diagnostic insights alongside teachers. Verify topological study plans properly prioritize root-cause topics. Deliverable Checkpoint: Teachers confirm generated root-cause gaps accurately reflect student learning needs.

Day 39 (Thu): First Scheduled Cloud Sync EventField Ops: Connect external 4G/satellite links to trigger outbox sync workers on pilot School Boxes. Confirm local student results sync to Central Cloud dashboards. Deliverable Checkpoint: Central Ministry and Regional dashboards reflect aggregate data from pilot schools.

Day 40 (Fri): Pilot Phase Evaluation & Structured FeedbackProduct: Conduct review sessions with school principals and lead teachers to collect structured qualitative feedback. Document minor calibration tweaks and UI enhancements for post-pilot updates.Deliverable Checkpoint: Formal pilot evaluation report completed with signed stakeholder feedback. 📊

Summary Operational Checklist

Month 1: Core Engine & Edge Readiness

- ✓ Week 1: Curriculum Ingestion & Graph Mirroring (Pillar 1)
- ✓ Week 2: Exam-CLO Matcher & ONNX Embeddings (Pillar 2)
- ✓ Week 3: Calibrated Gap Engine & Local LLM (Pillar 3)
- ✓ Week 4: Multi-School Tenant Isolation & Outbox Pattern

Month 2: Packaging, Simulation & Field Launch

- ✓ Week 5: School Box Image & Hardware Provisioning
- ✓ Week 6: 150-Device LAN Stress Test & Sync Dry-Run
- ✓ Week 7: On-Site Server Installation & Staff Training
- ✓ Week 8: Live Field Launch & Cloud Telemetry Sync